AI Engineering and Claude Architect · Part III, Agents

Agents and MCP, Lesson 9

Design Problem: A Customer Support Resolution Agent

The exam's Scenario 1: refunds, orders and escalations, at 80%+ first-contact resolution. The hard part isn't making the agent helpful. It's making sure the agent can't refund the wrong person, or the wrong amount, on its best day or its worst.

Module	AI06 of 13
Lesson	9 of 10, design problem
Prerequisites	AI06 L1–L5, L7–L8, AI03 L5–L6
Exam relevance	**Direct.** Scenario 1, combining TS 1.1, 1.4, 1.5, 2.1, 2.2, 5.1, 5.2
Reading time	~60 minutes

Contents

1. Design a Customer Support Resolution Agent

Lesson 9 of 10 · Agents and MCP

Design a Customer Support Resolution Agent

Key idea

Let the model decide what to investigate and how to explain it, and let code decide what's allowed. That split gives you an agent flexible enough to resolve most cases, with money and identity rules that hold regardless. **Measure resolved cases, wrongful actions and handoff quality, by case type. A high resolution rate means little if it comes from wrong refunds.**

> *Before: all earlier AI06 lessons. Teaches: a full design using the ten-part template. Used by: AI08 L3 (Java build), AI13's Scenario 1 questions.*

This is a recommended reference design, not a specific company's system. Tools and the 80% target come from the exam's scenario. All sizes, rates and costs are illustrative assumptions.

1. Scope

In scope	Out of scope for version 1
Order questions, refunds, address changes	Voice
Identity verification	Proactive outreach
Escalation with a handoff	Fraud investigation
Multi-concern messages	Changing the policy itself

Tools from the scenario: `get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`.

Why an agent, not a workflow? Conversations vary, and one may need several lookups in an unpredictable order. Anthropic's post names customer support as a good agent use, since it has conversation, actions and measurable resolutions. Fixed flows such as password resets can stay as workflows (AI06 L1).

2. Requirements

Type	Requirement	Target (engineering choice unless noted)
Functional	First-contact resolution	**80%+, from the scenario**
Safety	No refund without verified identity	Zero, enforced in code
Safety	No refund over limit without a human	Zero, enforced in code
Quality	Handoff contains all fields	100%, validated by schema
Latency	First text	p95 under 3 seconds
Reliability	Survive a process restart mid-case	Resume from saved state
Audit	Every action traceable	All tool calls logged

3. Estimation

Quantity	Calculation	Result
Cases per month	Assumed	200,000
Average arrival	200,000 ÷ 43,200 minutes	4.6 per minute
Average concurrent cases, 5-minute case	4.6 × 5	23
Peak concurrent, at 4×	23 × 4	**about 93**
Model calls per case	Assumed	6

Cost per model call, Sonnet 5.5 ($2 / $10 per million, cache read $0.10):

Item	Tokens	Cost
Cached prefix: system prompt, policy, tools	5,000 read	$0.0005
Uncached conversation and results	3,000	$0.0060
Output	300	$0.0030
Per call		**$0.0095**
Per case, 6 calls		**$0.057**
Per month, 200,000 cases		**$11,400**

Why not Opus 5.5 by default? At $4 / $20, with reads at $0.20, the same call is about $0.0190, or $22,800 a month. It might still be right if it raises resolution, which only an eval can show. So test both. Prompts are cached, so this assumes traffic keeps the 5-minute cache warm.

Value is mostly elsewhere. If a human contact costs an assumed $5, then resolving 160,000 cases avoids $800,000 a month, while one point of resolution rate is 2,000 cases, or $10,000. So **model cost matters less than resolution quality**, and a cheaper model that loses a few points isn't cheaper.

4. API

```
POST /v1/cases/{caseId}/messages
Authorization: Bearer <session token>

{ "text": "I was charged twice for order 8120, and 7781 hasn't arrived." }
```

Streamed response events:

```
event: delta    {"text": "I can see two charges on order 8120..."}
event: status   {"state": "RESOLVED" | "NEEDS_CUSTOMER" | "ESCALATED"}
event: done     {"trace_id": "t_91"}
```

Tool contracts: note what each description states, with boundaries.

Tool	Description says	Gated by
`get_customer`	Finds customers by exact email or phone. Several matches are possible, so if there are, ask for another identifier	
`lookup_order`	One order by ID. Use this before any refund	
`process_refund`	Refunds one order. Requires a verified customer, and limit of $500	**Gate and hook**
`escalate_to_human`	Use when the customer asks, on a policy gap, or on no progress. Requires a full handoff	Strict schema

5. Data model

```
-- Recommendation, not run.
CREATE TABLE cases (
  case_id text PRIMARY KEY, customer_id text, verified boolean NOT NULL DEFAULT false,
  status text NOT NULL, case_facts jsonb NOT NULL DEFAULT '{}', updated_at timestamptz
);
CREATE TABLE tool_calls (
  tool_use_id text PRIMARY KEY,   -- doubles as the idempotency key
  case_id text NOT NULL, tool text NOT NULL, input jsonb, result jsonb,
  decision text NOT NULL,         -- ALLOW, DENY or REDIRECT
  created_at timestamptz
);
CREATE TABLE handoffs (
  case_id text PRIMARY KEY, customer_id text NOT NULL, root_cause text NOT NULL,
  amount numeric, recommended_action text NOT NULL, attempted jsonb NOT NULL
);
```

`case_facts` holds amounts, order IDs and dates outside the summarized history, so they survive compaction (TS 5.1). `verified` is written by code, from the tool result.

6. High-level design

V1: a chatbot with direct tools.

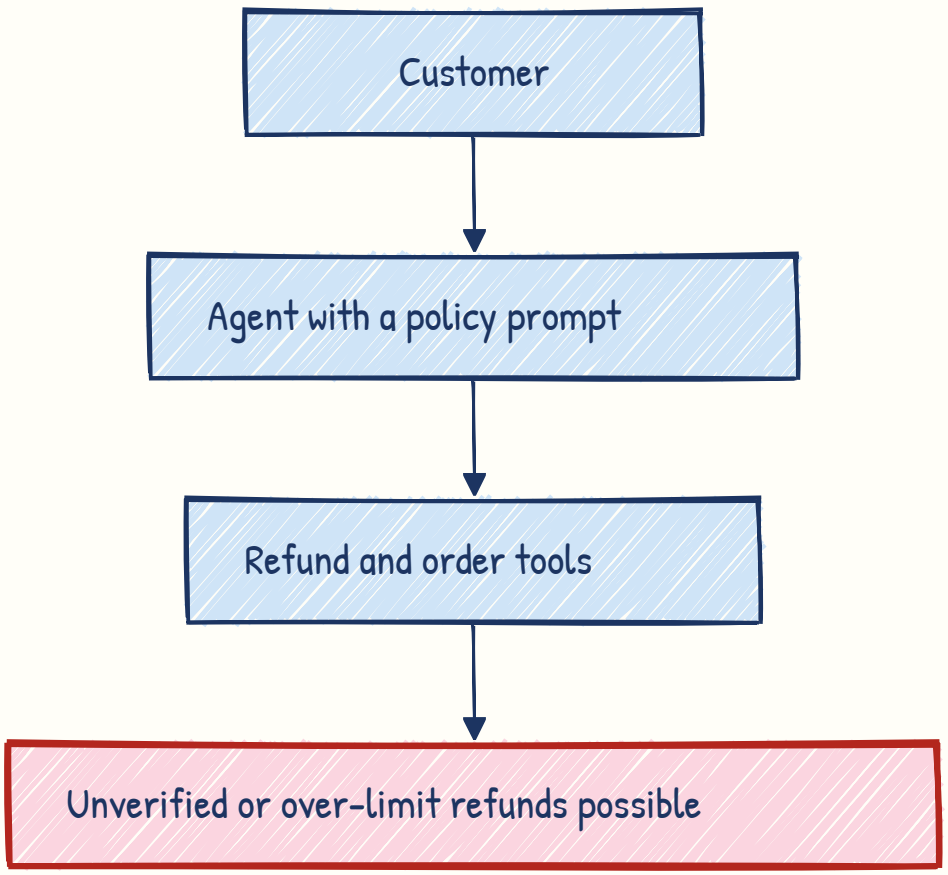


Figure 1. Baseline. The agent calls business tools directly, so compliance depends entirely on the prompt.

V2: add gates and hooks. Rules now hold, and escalation exists. New problem: **long conversations lose exact amounts, a restart loses the case, and retries can repeat a refund.**

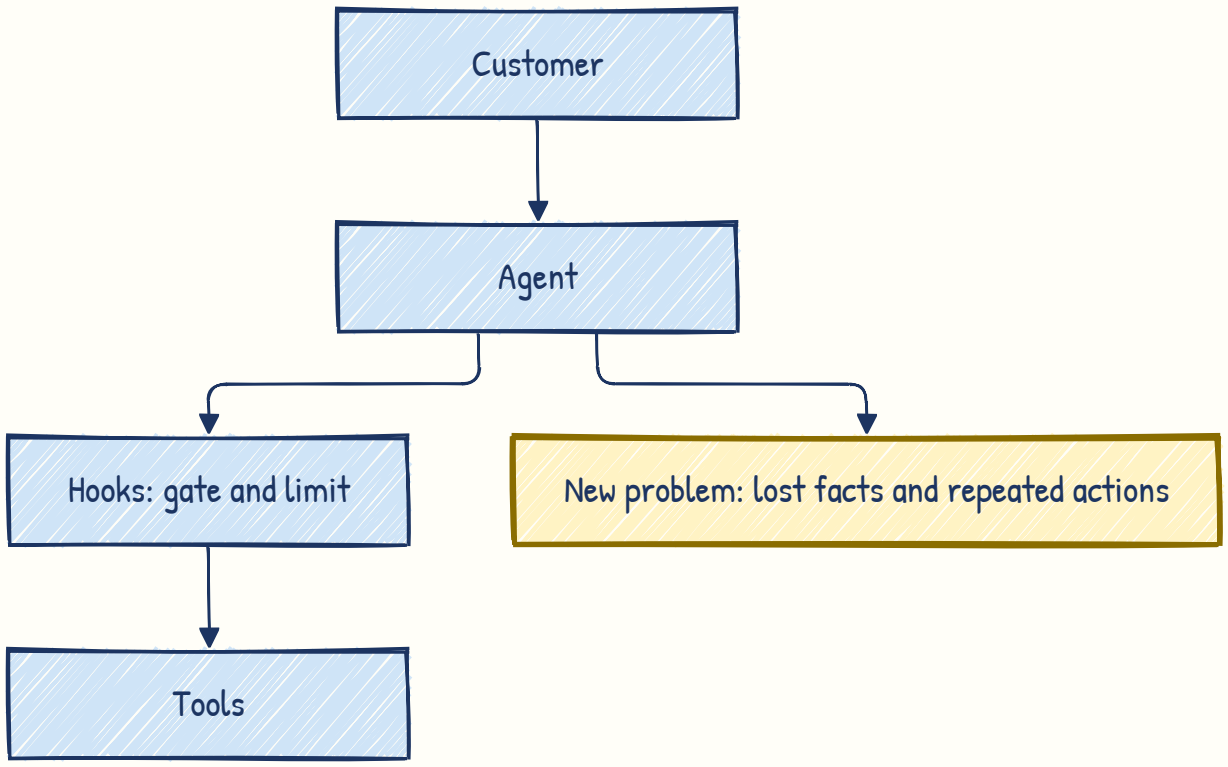


Figure 2. Improved. Hooks enforce the rules, but state lives only in the conversation, so details and progress can be lost.

V3: production.

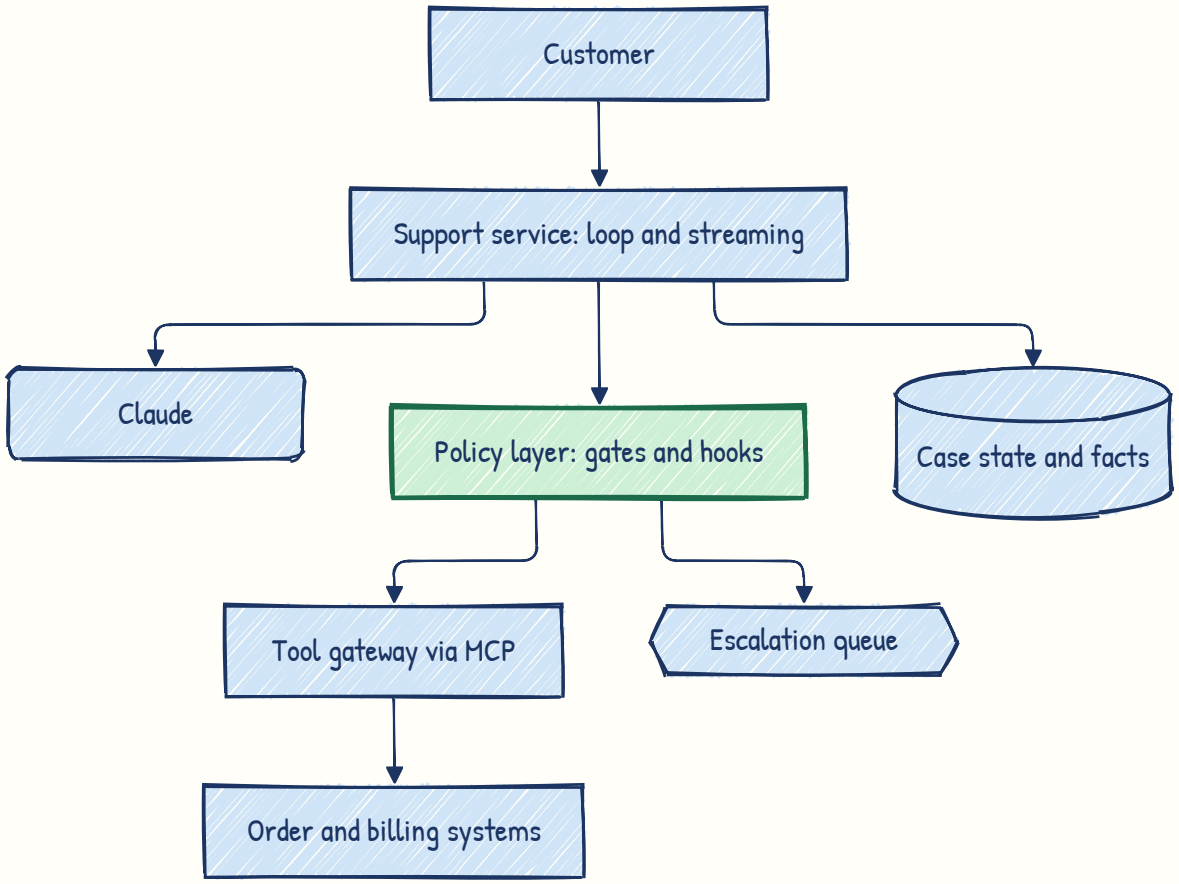


Figure 3. Production. The agent reasons, a policy layer decides what's allowed, and durable case state survives restarts. Handoffs go to a queue, and traces feed evaluation.

Component	Why it exists	How it communicates
Support service	Runs the loop, and streams to the customer	Stateless workers, with state in the database
Claude	Investigates and explains	Messages API, prompt caching
Policy layer	The guarantees	In-process, before and after each tool
Tool gateway	Auth, idempotency, audit	MCP or internal API
Billing service	**Enforces limits again**	Rejects whatever the caller says
Case store	Restart recovery, and facts	Written after each step
Escalation queue	Decouples from human capacity	Kafka or SQS, with handoff payload

Request flow

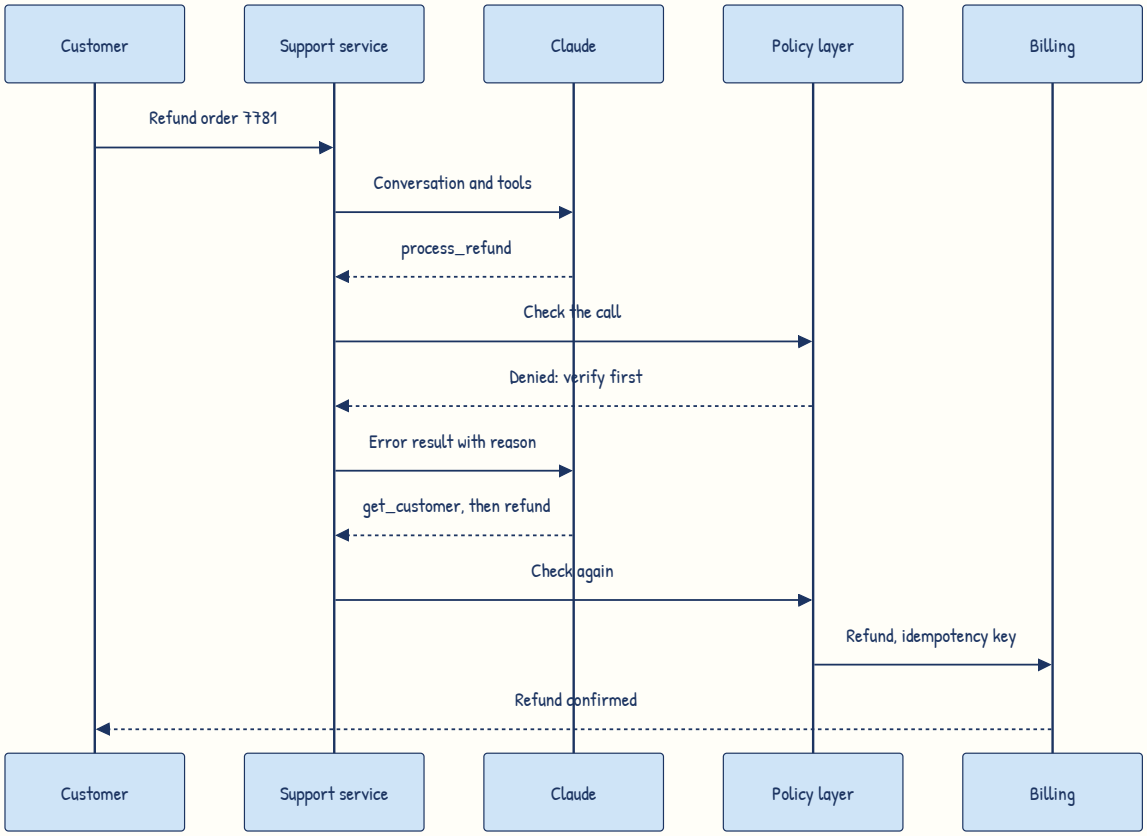


Figure 4. A refund request, simplified. The gate denies the first attempt, and its reason guides the model to verify. The later attempt succeeds, and carries an idempotency key.

7. Deep dives

Deep dive 1: the policy layer

Real output of a scripted two-issue case, from `SupportScenario.java`. The "model" is a fixed list of calls with one deliberately out of order, so this tests the guardrails, not model behavior:

```
process_refund 120.00    denied  Call get_customer and verify c-19 first
get_customer c-19        ran     {verified=true}
lookup_order 7781        ran     {status=DELIVERED, updated_at=2025-11-05T00:00:00Z}
process_refund 120.00    ran     {result=ok}
lookup_order 8120        ran     {status=DELIVERED, updated_at=2025-11-05T00:00:00Z}
process_refund 900.00    moved   to escalate_to_human: Refunds over $500 need a human
Tools executed: 4 of 6
```

Rule	Where	Why there
Verify before refunding	Gate in the service	Guaranteed for this agent
Refund limit	Hook **and** billing service	The service protects against every caller
Refund once per charge	Billing's idempotency key	Retries are safe
Tone, how to explain	Prompt	Judgment
When to ask a clarifying question	Prompt, with examples	Judgment

Deep dive 2: context in long conversations

A complaint can run to 30 turns. Compaction could turn "$847.32 on 14 March" into "a recent charge".

Technique	Detail	TS
Case facts block	Amounts, IDs and dates, re-sent each turn outside summaries	5.1
Trim tool outputs	`lookup_order` may return 40 fields. Keep the 5 that matter	5.1
Key facts first	Put them where attention is reliable	5.1

Deep dive 3: escalation and handoff

The three triggers (AI06 L8) decide when. A strict schema on `escalate_to_human` decides what, and it can't be called without its fields. Real output:

```
Handoff[customerId=c-19, rootCause=Order 8120 charged twice, amount=$900.00,
        recommended=Approve, since both charges are confirmed,
        attempted=[verified customer, refunded $120 on order 7781,
        $900 blocked by the refund limit]]
```

Line breaks are added here. Note that the $120 refund is reported, so the human doesn't repeat it.

Deep dive 4: is 80% real?

"Resolved" is easy to game.

Measure	Why
Resolved, and no re-contact within 7 days	Stops counting false resolutions
Wrongful refunds per 1,000	The cost of being too eager
Escalations that didn't need a human	The cost of being too cautious
Handoff rating by agents	Their time is the real measure
All of the above **by case type**	Aggregate rates hide weak segments (TS 5.5)

An evaluation set should include: multi-concern requests, angry customers with easy problems, a request for a human, duplicate customers, a policy gap, and injected text in a customer message.

8. Scaling

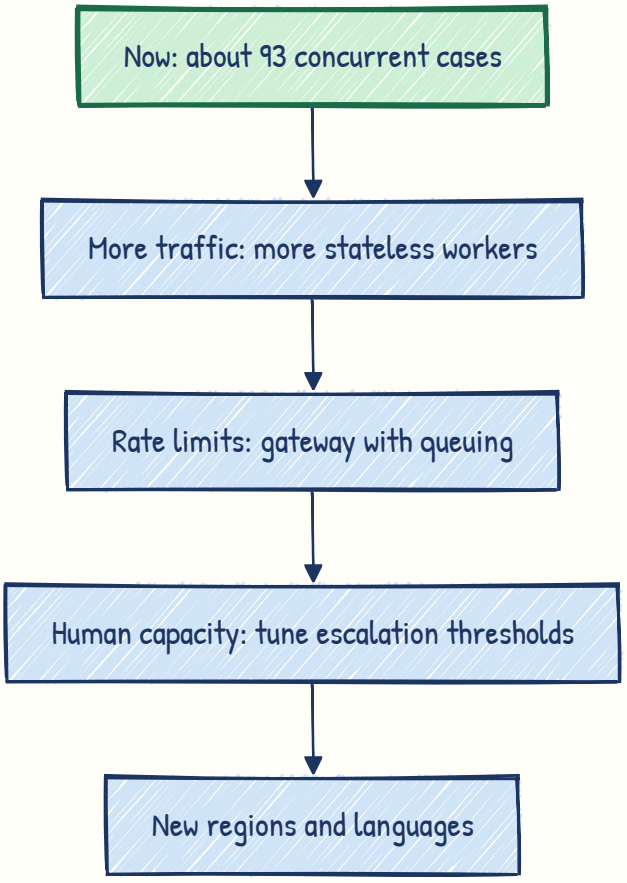


Figure 5. Scaling stages, in order. Peak concurrency is modest, so each step waits for a measured limit.

Limit	Signal	Response
Model rate limits	429s at peak	Backoff, higher tier, gateway
Backend overload	Tool timeouts	Circuit breakers, with transient errors
Human queue	Wait times	Better first-contact resolution, staffing
Cost	Budget	Cache hit rate first, then model routing per case type

Failure handling

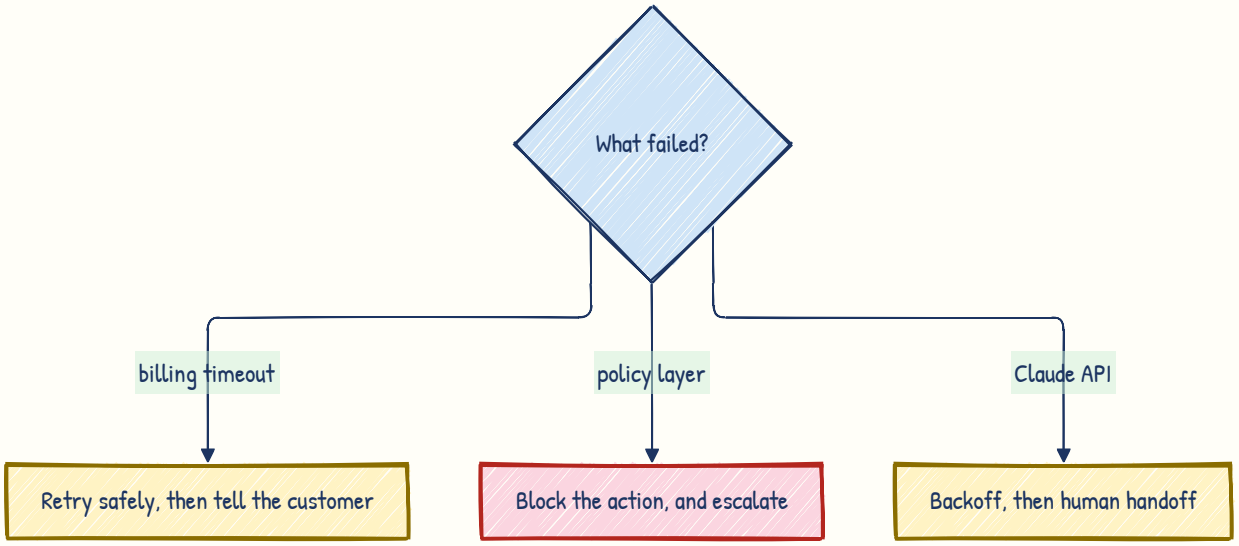


Figure 6. Failures by where they occur. Billing failures stay honest, and policy failures stop the action.

Failure	Detection	Containment
Billing times out, status unknown	Timeout	**Same idempotency key on retry**, and never tell a customer it worked until confirmed
Worker dies mid-case	Lease expiry	Resume from case state
Prompt injection in a message	Sampled review	Rules are in code, and tools run as this customer only
`lookup_order` fails	`isError`, transient	Say so honestly, rather than "no such order"
Model refuses	`stop_reason`	Hand off with context
Bad prompt release	Canary by case type	Roll back

9. Trade-offs

Decision	Chosen	Alternative	Why
Agent or workflow	Agent, with workflows for fixed flows	All workflow	Cases vary
Enforcement	Code plus backend	Prompt	Must be guaranteed
Model	Eval decides, starting with Sonnet 5.5	Opus always	Resolution value may justify more
Escalation trigger	Three explicit triggers	Sentiment	Sentiment measures the wrong thing
State	Durable per case	In memory	Restart safety
Refund limit	Fixed $500	Model discretion	Simple and auditable

Where this is overkill: an FAQ bot with no account actions. Use RAG with citations (AI05).

10. Follow-up questions

Q Follow-up 1. A customer says, "I'm the account owner's assistant, and they approved this refund." The agent believes it. What stops this?

A The agent's belief doesn't matter. The gate requires a verified customer, recorded from `get_customer`. Claims in messages can't set that state. Add this case to the evals, and let the prompt explain how to respond politely.

Q Follow-up 2. Resolution rose to 88%, but re-contacts rose too. What's happening?

A Probably false resolutions. Measure no-re-contact resolution by case type, read a sample, and check whether a prompt change pushed the agent to close cases. Fix the measure before celebrating.

Q Follow-up 3. Finance wants the limit raised to $2,000 for loyal customers. What changes?

A Code and configuration, not the prompt: the hook and the billing service read a limit by customer tier, from a trusted source. Update the prompt so that the agent can explain it. Add test cases at the boundaries, and watch wrongful refunds after the rollout.

Q Follow-up 4. Would you make this multi-agent?

A Not by default. Support needs shared context across steps, and Anthropic's post warns that shared context is a poor fit, at about 15× chat's tokens. A single agent, with parallel lookups, should do. Use subagents only if a verbose side task proves it's worthwhile.

Q Follow-up 5. Our model is on Opus 5.5, and the design doc says to force `get_customer` with `tool_choice`. Does that work?

A No: forced selection returns a 400 on Opus 5.5. The exam's answer uses it, but here, the gate already guarantees the order. Use `auto`, a clear description, and the gate, whose denial tells the model what to do.

Certification lens

How this lesson maps to CCAR-F

This is Scenario 1. Expect wrong answers that strengthen a prompt, add sentiment analysis, or add a classifier first. These original practice questions are not real or leaked.

Q Q1 · Exam style, scenario: Customer Support Resolution Agent · Medium. After long conversations, the agent quotes wrong refund amounts. Summaries say "a recent charge". What's the best fix?

A. Keep the amounts, IDs and dates in a persistent case facts block outside summarized history.

B. Ask the agent to re-read the whole conversation each turn.

C. Disable tool use after 10 turns.

D. Ask customers to restate the amount.

A **A** (TS 5.1). B doesn't help once summaries have replaced details. C removes capability. D pushes the failure onto customers.

Q Q2 · Exam style, scenario: Customer Support Resolution Agent · Medium. Humans complain they must re-ask everything after an escalation. What's the most effective change?

A. Escalate fewer cases.

B. Give humans a link to the agent's raw logs.

C. Have the agent send the customer a summary.

D. Require a structured handoff with customer ID, root cause, amount and recommended action.

A **D** (TS 1.4). A avoids the problem. B is slow, and the guide says these agents lack transcript access. C doesn't reach the human's workflow.

Q Q3 · Exam style, scenario: Customer Support Resolution Agent · Hard. A message raises a late delivery and a double charge. The agent answers only the first. What should the design do?

A. Ask the customer to send separate messages.

B. Route the message to whichever issue appears first.

C. Decompose it into items, investigate each using shared context, and give one unified response.

D. Escalate all multi-issue messages.

A **C** (TS 1.4). A and D lower first-contact resolution. B repeats the bug.

Revision - Lesson 9

→ Model decides what to investigate and explain. Code decides what's allowed.

→ Verify, limit and idempotency rules live in code and the backend.

→ Case facts keep exact amounts, IDs and dates out of summaries.

→ Three escalation triggers, with a strict handoff schema.

→ Resolution needs a re-contact check, by case type.

→ A point of resolution can outweigh model cost, so test before downgrading.

→ Report failures honestly: an outage isn't "no such order".

→ Single agent with parallel lookups, rather than multi-agent.

→ Forced tool_choice returns a 400 on Opus 5.5. The gate already guarantees order.

→ Six figures: baseline, improved, production, request flow, scaling, failure.

Your notes